

```

import os
import json
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split

dataset_dir = '../dataset'

# 13 Skema Label
list_label = [
    'PRODUCT_POSITIVE', 'PRODUCT_NEGATIVE', 'PRODUCT_NEUTRAL',
    'PRICE_POSITIVE', 'PRICE_NEGATIVE', 'PRICE_NEUTRAL',
    'PLACE_POSITIVE', 'PLACE_NEGATIVE', 'PLACE_NEUTRAL',
    'PROMOTION_POSITIVE', 'PROMOTION_NEGATIVE', 'PROMOTION_NEUTRAL',
    'OUT_OF_TOPIC'
]

print(" Membaca JSONL & Membuat Kolom Multilabel (One-Hot
Encoding)...")
jsonl_path = os.path.join(dataset_dir, 'Kelp3_dataset_anotasi.jsonl')

if not os.path.exists(jsonl_path):
    print(f"Error: File {jsonl_path} tidak ditemukan! Pastikan
posisinya benar.")
else:
    with open(jsonl_path, 'r', encoding='utf-8') as f:
        data_ner_master = [json.loads(line) for line in f]

    # Ekstrak data untuk keperluan tabel biner Multilabel
    records_ml = []
    for data in data_ner_master:
        text = data.get('text', '')
        labels = data.get('accept', [])

        row = {'cleaned_text': text}
        for lbl in list_label:
            row[lbl] = 1 if lbl in labels else 0
        records_ml.append(row)

    df_multilabel_master = pd.DataFrame(records_ml)
    print("    -> Selesai mengekstrak data teks dan 13 komponen label
biner.")

    # Memotong Data Proyek A (Multilabel Text Classification)
    print("\nMelakukan Data Splitting Proyek A (CSV)...")
    train_ml, temp_ml = train_test_split(df_multilabel_master,
test_size=0.30, random_state=42)
    val_ml, test_ml = train_test_split(temp_ml, test_size=0.50,

```

```

random_state=42)

    # Simpan berkas CSV baru yang sudah lengkap dengan 13 kolom label
    train_ml.to_csv(os.path.join(dataset_dir,
    'Kelp3_multilabel_train.csv'), index=False)
    val_ml.to_csv(os.path.join(dataset_dir,
    'Kelp3_multilabel_val.csv'), index=False)
    test_ml.to_csv(os.path.join(dataset_dir,
    'Kelp3_multilabel_test.csv'), index=False)
    print(f"    Proyek A Sukses! Train: {len(train_ml)}, Val:
    {len(val_ml)}, Test: {len(test_ml)}")

    # Memotong Data Proyek B (Named Entity Recognition)
    print("\nMelakukan Data Splitting Proyek B (JSONL)...")
    # random_state=42 yang sama persis agar baris urutannya sinkron
    train_ner, temp_ner = train_test_split(data_ner_master,
    test_size=0.30, random_state=42)
    val_ner, test_ner = train_test_split(temp_ner, test_size=0.50,
    random_state=42)

    # Helper function untuk menyimpan file format JSONL
    def save_jsonl(data, filename):
        output_path = os.path.join(dataset_dir, filename)
        with open(output_path, 'w', encoding='utf-8') as out_f:
            for item in data:
                out_f.write(json.dumps(item, ensure_ascii=False) + '\n')

    # Simpan berkas JSONL hasil pemotongan
    save_jsonl(train_ner, 'Kelp3_ner_train.jsonl')
    save_jsonl(val_ner, 'Kelp3_ner_val.jsonl')
    save_jsonl(test_ner, 'Kelp3_ner_test.jsonl')
    print(f"    Proyek B Sukses! Train: {len(train_ner)}, Val:
    {len(val_ner)}, Test: {len(test_ner)}")
    print("\nSemua 6 file split data wajib di folder dataset!")

    # Menggambar Grafik EDA (Kriteria Penilaian Nilai 95)
    print("\nMenggambar Grafik EDA Proyek A...")
    sns.set_theme(style="whitegrid")
    fig, axes = plt.subplots(2, 2, figsize=(18, 14))

    # GRAPH 1: Distribusi Panjang Karakter Review
    train_ml['char_length'] =
    train_ml['cleaned_text'].astype(str).apply(len)
    sns.histplot(train_ml['char_length'], bins=30, kde=True,
    ax=axes[0, 0], color='skyblue')
    axes[0, 0].set_title('Distribusi Panjang Karakter Review',
    fontsize=14)
    axes[0, 0].set_xlabel('Jumlah Karakter')
    axes[0, 0].set_ylabel('Frekuensi')

```

```

# GRAPH 2: Distribusi Jumlah Kata/Token per Review
train_ml['word_count'] =
train_ml['cleaned_text'].astype(str).apply(lambda x: len(x.split()))
sns.histplot(train_ml['word_count'], bins=30, kde=True, ax=axes[0,
1], color='salmon')
axes[0, 1].set_title('Distribusi Jumlah Kata/Token per Review',
fontsize=14)
axes[0, 1].set_xlabel('Jumlah Kata')
axes[0, 1].set_ylabel('Frekuensi')

# GRAPH 3: Distribusi Kemunculan Setiap Label ABSA (Mendeteksi
Imbalance)
label_counts =
train_ml[list_label].sum().sort_values(ascending=False)
sns.barplot(x=label_counts.values, y=label_counts.index,
ax=axes[1, 0], palette='viridis')
axes[1, 0].set_title('Distribusi Total Frekuensi Label ABSA',
fontsize=14)
axes[1, 0].set_xlabel('Jumlah Kemunculan')
axes[1, 0].set_ylabel('Label ABSA')

# GRAPH 4: Correlation Matrix antar Label (Melihat Hubungan Aspek)
corr_matrix = train_ml[list_label].corr()
sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap='coolwarm',
ax=axes[1, 1],
cbar=True, annot_kws={"size": 8},
xticklabels=list_label, yticklabels=list_label)
axes[1, 1].set_title('Correlation Matrix Antar Label ABSA',
fontsize=14)
plt.setp(axes[1, 1].get_xticklabels(), rotation=45, ha="right",
rotation_mode="anchor")

plt.tight_layout()
plt.show()
print("Pemrosesan data dan visualisasi selesai")

```

Membaca JSONL & Membuat Kolom Multilabel (One-Hot Encoding)...

-> Selesai mengekstrak data teks dan 13 komponen label biner.

Melakukan Data Splitting Proyek A (CSV)...

Proyek A Sukses! Train: 2218, Val: 475, Test: 476

Melakukan Data Splitting Proyek B (JSONL)...

Proyek B Sukses! Train: 2218, Val: 475, Test: 476

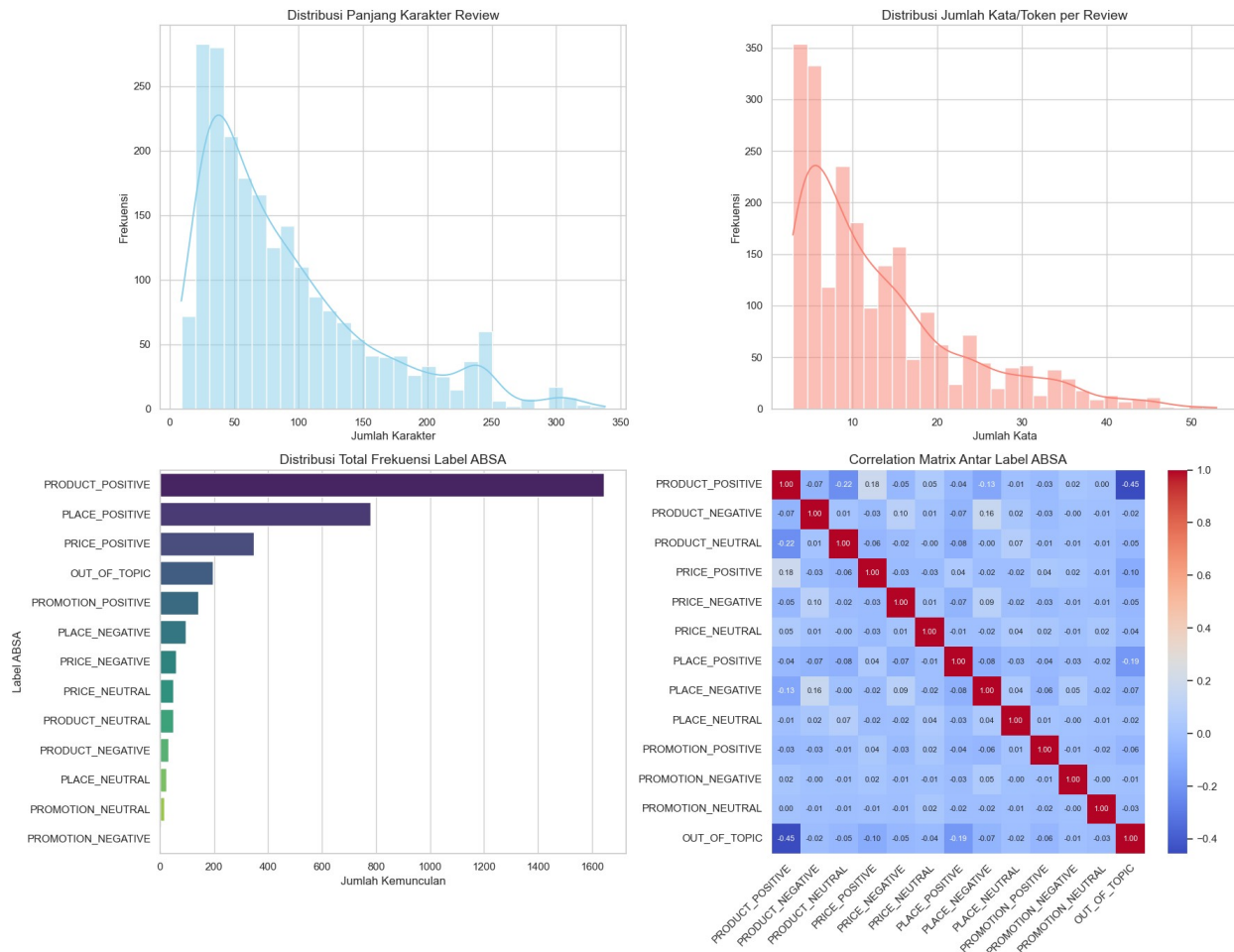
Semua 6 file split data wajib di folder dataset!

Menggambar Grafik EDA Proyek A...

```
C:\Users\PC\AppData\Local\Temp\ipykernel_28940\3784723486.py:95:
FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `y` variable to `hue` and set
`legend=False` for the same effect.
```

```
sns.barplot(x=label_counts.values, y=label_counts.index, ax=axes[1,
0], palette='viridis')
```



Pemrosesan data dan visualisasi selesai

```
import re
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer

print("Membaca data train, validation, dan test untuk Proyek A...")
# Load data yang sudah di-split di Step 2 sebelumnya
train_df = pd.read_csv('../dataset/Kelp3_multilabel_train.csv')
val_df = pd.read_csv('../dataset/Kelp3_multilabel_val.csv')
```

```

test_df = pd.read_csv('../dataset/Kelp3_multilabel_test.csv')

# DATA PREPROCESSING (CLEANING FUNCTION)
def clean_indonesian_text(text):
    if pd.isna(text):
        return ""
    # 1. Case folding (mengubah semua huruf menjadi kecil)
    text = str(text).lower()
    # 2. Menghapus tautan/URL
    text = re.sub(r'https?://\s+|www\.\s+', '', text)
    # 3. Menghapus karakter aneh, emoji, dan tanda baca (hanya
    menyisakan huruf, angka, dan spasi)
    text = re.sub(r'[^a-zA-Z0-9\s]', ' ', text)
    # 4. Menghapus spasi ganda yang berlebihan
    text = re.sub(r'\s+', ' ', text).strip()
    return text

print("\nMenjalankan proses Preprocessing / Cleaning Teks...")
# nerapin fungsi cleaning ke semua pembagian data
train_df['clean_text'] =
train_df['cleaned_text'].apply(clean_indonesian_text)
val_df['clean_text'] =
val_df['cleaned_text'].apply(clean_indonesian_text)
test_df['clean_text'] =
test_df['cleaned_text'].apply(clean_indonesian_text)

print("Contoh hasil sebelum vs sesudah preprocessing:")
print(f"Sebelum: {train_df['cleaned_text'].iloc[1][:90]}...")
print(f"Sesudah: {train_df['clean_text'].iloc[1][:90]}...")

# FEATURE ENGINEERING (REPRESENTASI TEKS 1 - TF-IDF N-GRAM)
print("\nMembuat Representasi Teks Pertama: TF-IDF (Unigram +
Bigram)...")

# Inisialisasi TF-IDF Vectorizer
# ngram_range=(1, 2) artinya kita mengambil kata tunggal (unigram) dan
kombinasi 2 kata berurutan (bigram)
tfidf_vectorizer = TfidfVectorizer(ngram_range=(1, 2),
max_features=5000)

# Fit ke data train, lalu transform ke train, val, dan test
X_train_tfidf = tfidf_vectorizer.fit_transform(train_df['clean_text'])
X_val_tfidf = tfidf_vectorizer.transform(val_df['clean_text'])
X_test_tfidf = tfidf_vectorizer.transform(test_df['clean_text'])

print(f"Matriks TF-IDF Train berhasil dibuat dengan ukuran:
{X_train_tfidf.shape}")
print("Sukses! Langkah Preprocessing dan Representasi Teks Pertama
telah selesai.")

```

Membaca data train, validation, dan test untuk Proyek A...

Menjalankan proses Preprocessing / Cleaning Teks...

Contoh hasil sebelum vs sesudah preprocessing:

Sebelum: Pas buat emak emak yg suka shopping.....

Sesudah: pas buat emak emak yg suka shopping...

Membuat Representasi Teks Pertama: TF-IDF (Unigram + Bigram)...

Matriks TF-IDF Train berhasil dibuat dengan ukuran: (2218, 5000)

Sukses! Langkah Preprocessing dan Representasi Teks Pertama telah selesai.

```
import pandas as pd
from sklearn.multioutput import MultiOutputClassifier, ClassifierChain
from sklearn.svm import LinearSVC
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score, hamming_loss, accuracy_score

# 1. menyiapkan data target label (Y) untuk Train dan Validation
list_label = [
    'PRODUCT_POSITIVE', 'PRODUCT_NEGATIVE', 'PRODUCT_NEUTRAL',
    'PRICE_POSITIVE', 'PRICE_NEGATIVE', 'PRICE_NEUTRAL',
    'PLACE_POSITIVE', 'PLACE_NEGATIVE', 'PLACE_NEUTRAL',
    'PROMOTION_POSITIVE', 'PROMOTION_NEGATIVE', 'PROMOTION_NEUTRAL',
    'OUT_OF_TOPIC'
]

y_train = train_df[list_label]
y_val = val_df[list_label]

# 2. mendefinisikan 2 Algoritma ML dan 2 Teknik Transformasi Masalah
# pakai LinearSVC (SVM) dan Logistic Regression karena sangat cepat &
# kuat untuk TF-IDF
models_pool = {
    'SVM (LinearSVC)': LinearSVC(random_state=42,
    class_weight='balanced'),
    'Logistic Regression': LogisticRegression(random_state=42,
    class_weight='balanced', max_iter=1000)
}

print("Memulai eksperimen pelatihan model multilabel...")
eksperimen_results = []

# Loop untuk menggabungkan Algoritma dan Problem Transformation
for algo_name, algo_obj in models_pool.items():

    # TEKNIK 1 BINARY RELEVANCE (MultiOutputClassifier)
    br_model = MultiOutputClassifier(algo_obj)
    br_model.fit(X_train_tfidf, y_train)
    br_pred = br_model.predict(X_val_tfidf)
```

```

eksperimen_results.append({
    'Pendekatan': 'Binary Relevance',
    'Algoritma': algo_name,
    'Subset Accuracy (Strict)': accuracy_score(y_val, br_pred),
    'Hamming Loss': hamming_loss(y_val, br_pred),
    'Macro F1-Score': f1_score(y_val, br_pred, average='macro'),
    'Micro F1-Score': f1_score(y_val, br_pred, average='micro')
})

# TEKNIK 2 CLASSIFIER CHAINS
cc_model = ClassifierChain(algo_obj, order='random',
random_state=42)
cc_model.fit(X_train_tfidf, y_train)
cc_pred = cc_model.predict(X_val_tfidf)

eksperimen_results.append({
    'Pendekatan': 'Classifier Chains',
    'Algoritma': algo_name,
    'Subset Accuracy (Strict)': accuracy_score(y_val, cc_pred),
    'Hamming Loss': hamming_loss(y_val, cc_pred),
    'Macro F1-Score': f1_score(y_val, cc_pred, average='macro'),
    'Micro F1-Score': f1_score(y_val, cc_pred, average='micro')
})

# 3. Rekap hasil ke dalam DataFrame
df_metrik_uas = pd.DataFrame(eksperimen_results)

print("\nHASIL PERBANDINGAN EKSPERIMEN MODEL MULTILABEL (DATA
VALIDASI):")
display(df_metrik_uas.sort_values(by='Macro F1-Score',
ascending=False))

```

Memulai eksperimen pelatihan model multilabel...

HASIL PERBANDINGAN EKSPERIMEN MODEL MULTILABEL (DATA VALIDASI):

```

{"columns":[{"name":"index","rawType":"int64","type":"integer"},
{"name":"Pendekatan","rawType":"object","type":"string"},
{"name":"Algoritma","rawType":"object","type":"string"},
{"name":"Subset Accuracy
(Strict)","rawType":"float64","type":"float"},{"name":"Hamming
Loss","rawType":"float64","type":"float"},{"name":"Macro F1-
Score","rawType":"float64","type":"float"},{"name":"Micro F1-
Score","rawType":"float64","type":"float"}],"ref":"8795c524-40b3-4930-
9d91-b38558e5dc79","rows":[["2","Binary Relevance","Logistic
Regression","0.4442105263157895","0.06591093117408907","0.426257489889
5601","0.7375886524822695"],["1","Classifier Chains","SVM
(LinearSVC)","0.4694736842105263","0.06769230769230769","0.41320806490
837053","0.7231788079470198"],["0","Binary Relevance","SVM

```

```

(LinearSVC)","0.4610526315789474","0.06866396761133603","0.41073050254
3782","0.7217847769028871"],["3","Classifier Chains","Logistic
Regression","0.45052631578947366","0.07611336032388664","0.38767323573
979834","0.6932114882506527"]], "shape":{"columns":6,"rows":4}}

import numpy as np
from sklearn.metrics import classification_report, hamming_loss,
accuracy_score

# 1. Ambil target label asli untuk data test
y_test = test_df[list_label]

print("Melatih ulang model terbaik pada data Train untuk dievaluasi ke
Test Set...")
# menggunakan kombinasi terbaik Binary Relevance + Logistic Regression
best_model = MultiOutputClassifier(LogisticRegression(random_state=42,
class_weight='balanced', max_iter=1000))
best_model.fit(X_train_tfidf, y_train)

# 2. Prediksi ke Data Test
y_pred_test = best_model.predict(X_test_tfidf)

# 3. menampilkan Metrik Evaluasi Global di Test Set
print("\n=====")
print("HASIL EVALUASI MODEL TERBAIK PADA TEST SET")
print("=====")
print(f"Subset Accuracy (Strict) : {accuracy_score(y_test,
y_pred_test):.4f}")
print(f"Hamming Loss : {hamming_loss(y_test,
y_pred_test):.4f}")
print("=====\\n")

# 4. menampilkan Classification Report per Label
print("CLASSIFICATION REPORT PER LABEL:")
print(classification_report(y_test, y_pred_test,
target_names=list_label, zero_division=0))

# 5. KODE ANALISIS KESALAHAN (ERROR ANALYSIS)
print("\\nMEMULAI ANALISIS KESALAHAN MODEL (CONTOH KASUS SALAH
PREDIKSI):")
print("=====
=====")

# mencari baris mana saja yang hasil prediksinya tidak sama persis
dengan label asli
salah_indeks = np.where(np.any(y_test.values != y_pred_test, axis=1))
[0]

count = 0
for idx in salah_indeks:

```

```

if count >= 3: # menampilkan 3 contoh
    break

# Ambil teks asli sebelum di-clean agar mudah dibaca di laporan
teks_asli = test_df['cleaned_text'].iloc[idx]

# Ambil label asli yang bernilai 1
label_asli_aktif = [list_label[i] for i, val in
enumerate(y_test.values[idx]) if val == 1]
# Ambil label hasil prediksi yang bernilai 1
label_pred_aktif = [list_label[i] for i, val in
enumerate(y_pred_test[idx]) if val == 1]

print(f"\nContoh Salah {count+1}:")
print(f"    Teks Review : '{teks_asli[:150]}...'")
print(f"    Label Asli   : {label_asli_aktif}")
print(f"    AI Menebak   : {label_pred_aktif}")
print("-" * 68)
count += 1

```

Melatih ulang model terbaik pada data Train untuk dievaluasi ke Test Set...

```

=====
HASIL EVALUASI MODEL TERBAIK PADA TEST SET
=====
Subset Accuracy (Strict) : 0.4769
Hamming Loss           : 0.0682
=====

```

CLASSIFICATION REPORT PER LABEL:

	precision	recall	f1-score	support
PRODUCT_POSITIVE	0.91	0.87	0.89	367
PRODUCT_NEGATIVE	0.50	0.45	0.48	11
PRODUCT_NEUTRAL	0.14	0.29	0.19	14
PRICE_POSITIVE	0.84	0.94	0.88	77
PRICE_NEGATIVE	0.56	0.82	0.67	11
PRICE_NEUTRAL	0.31	0.42	0.36	12
PLACE_POSITIVE	0.68	0.70	0.69	167
PLACE_NEGATIVE	0.78	0.82	0.80	22
PLACE_NEUTRAL	0.10	0.20	0.13	5
PROMOTION_POSITIVE	0.16	0.40	0.23	25
PROMOTION_NEGATIVE	0.00	0.00	0.00	4
PROMOTION_NEUTRAL	0.00	0.00	0.00	2
OUT_OF_TOPIC	0.33	0.50	0.40	32
micro avg	0.70	0.77	0.73	749
macro avg	0.41	0.49	0.44	749
weighted avg	0.75	0.77	0.76	749

samples avg	0.71	0.77	0.72	749
-------------	------	------	------	-----

MEMULAI ANALISIS KESALAHAN MODEL (CONTOH KASUS SALAH PREDIKSI):

Contoh Salah 1:

Teks Review : 'Nice product with a suitable price.....'
Label Asli : ['PRODUCT_POSITIVE', 'PRICE_POSITIVE']
AI Menebak : ['PRICE_POSITIVE']

Contoh Salah 2:

Teks Review : 'Koleksinya banyak wayang. Dan harganya sedikit mahal. Tapi kalau yang mau cari barang otentik, ini salah satu rekomendasi bagus buat dicoba....'
Label Asli : ['PRODUCT_POSITIVE', 'PRODUCT_NEGATIVE', 'PRICE_NEGATIVE']
AI Menebak : ['PRODUCT_POSITIVE', 'PRICE_NEGATIVE']

Contoh Salah 3:

Teks Review : 'Puas belanja disini..semoga bisa balik sini lagi..amiinn...'
Label Asli : ['PRODUCT_POSITIVE']
AI Menebak : ['PLACE_POSITIVE']

```
import joblib
import os
```

```
os.makedirs('../models', exist_ok=True)
```

```
# Simpan model terbaik dan vectorizer TF-IDF-nya
```

```
joblib.dump(best_model, '../models/Kelp3_best_multilabel_model.pkl')
joblib.dump(tfidf_vectorizer, '../models/Kelp3_tfidf_vectorizer.pkl')
```

```
print("Model klasifikasi multilabel dan TF-IDF Vectorizer berhasil  
disimpan di folder models/ !")
```

Model klasifikasi multilabel dan TF-IDF Vectorizer berhasil disimpan di folder models/ !

```

import os
import json
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

dataset_dir = '../dataset'
ner_train_path = os.path.join(dataset_dir, 'Kelp3_ner_train.jsonl')

tokens_per_review = []
entities_per_review = []
all_entity_labels = []
span_word_counts = []

print("Membaca berkas JSONL dan mengekstrak komponen NER...")

with open(ner_train_path, 'r', encoding='utf-8') as f:
    for line in f:
        data = json.loads(line)
        text = data.get('text', '')
        tokens = data.get('tokens', [])
        spans = data.get('spans', [])

        # 1. Hitung jumlah token per review
        tokens_per_review.append(len(tokens))

        # 2. Hitung jumlah entitas/span per review
        entities_per_review.append(len(spans))

        # 3. liat setiap entitas di dalam teks
        for span in spans:
            label = span.get('label', 'UNKNOWN')
            all_entity_labels.append(label)

            # Potong teks asli berdasarkan koordinat start & end span
            start_char = span.get('start', 0)
            end_char = span.get('end', 0)
            extracted_span_text = text[start_char:end_char]

            # Hitung berapa kata di dalam span tersebut
            word_count = len(extracted_span_text.split())
            span_word_counts.append({
                'Label': label,
                'Word_Count': word_count
            })

# Ubah ke DataFrame agar mudah digambar
df_review_metrics = pd.DataFrame({
    'Tokens_Count': tokens_per_review,

```

```

        'Entities_Count': entities_per_review
    })
df_span_metrics = pd.DataFrame(span_word_counts)

print("Ekstrak data NER selesai!")

# PROSES MENGGAMBAR 4 GRAFIK EDA NER (TARGET NILAI 95)
print("\nMenggambar Grafik EDA Proyek B...")
sns.set_theme(style="whitegrid")
fig, axes = plt.subplots(2, 2, figsize=(18, 14))

# GRAPH 1: Distribusi Jumlah Token per Review
sns.histplot(df_review_metrics['Tokens_Count'], bins=25, kde=True,
ax=axes[0, 0], color='teal')
axes[0, 0].set_title('Distribusi Jumlah Token per Review',
    fontsize=14)
axes[0, 0].set_xlabel('Jumlah Token')
axes[0, 0].set_ylabel('Frekuensi')

# GRAPH 2: Jumlah Entitas per Review
# Menggunakan countplot karena jumlah entitas per review biasanya
# berupa angka bulat kecil
sns.countplot(x='Entities_Count', data=df_review_metrics, ax=axes[0,
    1], palette='muted')
axes[0, 1].set_title('Jumlah Entitas (Span) per Review', fontsize=14)
axes[0, 1].set_xlabel('Jumlah Entitas di Dalam Satu Review')
axes[0, 1].set_ylabel('Frekuensi Review')

# GRAPH 3: Total Entitas Tiap Label (Distribusi Kelas NER)
label_series = pd.Series(all_entity_labels).value_counts()
sns.barplot(x=label_series.values, y=label_series.index, ax=axes[1,
    0], palette='magma')
axes[1, 0].set_title('Total Entitas Tiap Label Aspek', fontsize=14)
axes[1, 0].set_xlabel('Total Kemunculan Span')
axes[1, 0].set_ylabel('Label Entitas')

# GRAPH 4: Rata-rata Jumlah Kata/Token per Jenis Entitas
sns.barplot(x='Word_Count', y='Label', data=df_span_metrics,
ax=axes[1, 1], palette='plasma', errorbar=None)
axes[1, 1].set_title('Rata-rata Jumlah Kata per Jenis Entitas/Span',
    fontsize=14)
axes[1, 1].set_xlabel('Rata-rata Jumlah Kata')
axes[1, 1].set_ylabel('Label Entitas')

plt.tight_layout()
plt.show()
print("Selesai!")

```

Membaca berkas JSONL dan mengekstrak komponen NER...
 Ekstrak data NER selesai!

Menggambar Grafik EDA Proyek B...

C:\Users\PC\AppData\Local\Temp\ipykernel_31576\1915576908.py:70:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x='Entities_Count', data=df_review_metrics, ax=axes[0, 1], palette='muted')
```

C:\Users\PC\AppData\Local\Temp\ipykernel_31576\1915576908.py:77:
FutureWarning:

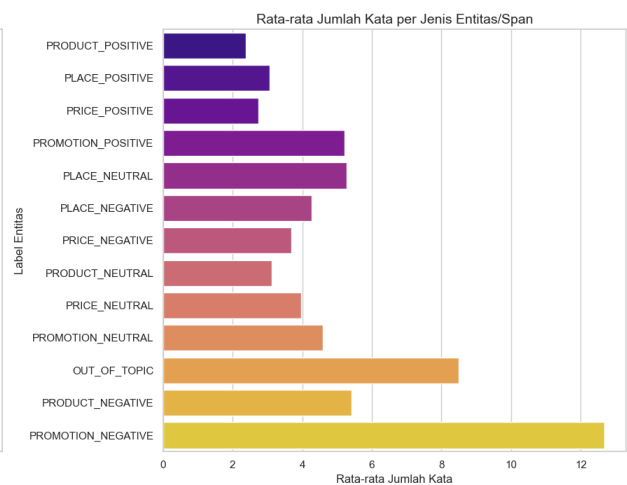
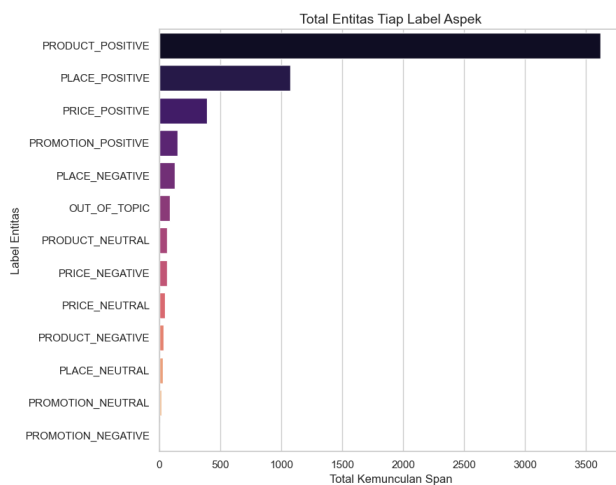
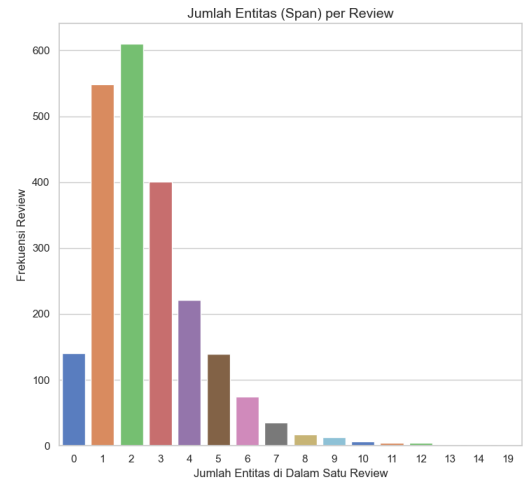
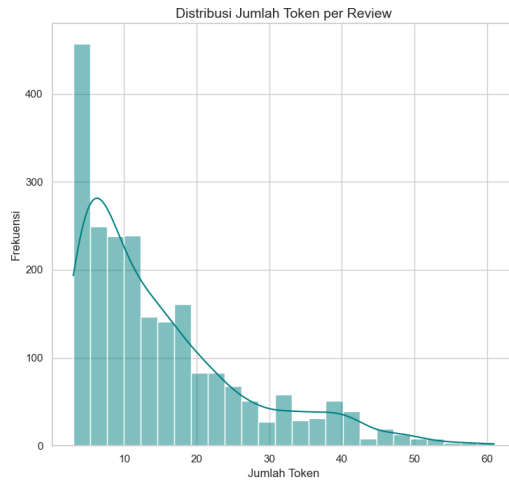
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x=label_series.values, y=label_series.index, ax=axes[1, 0], palette='magma')
```

C:\Users\PC\AppData\Local\Temp\ipykernel_31576\1915576908.py:83:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x='Word_Count', y='Label', data=df_span_metrics, ax=axes[1, 1], palette='plasma', errorbar=None)
```



Selesai!

```
import os
import json
import pandas as pd

dataset_dir = '../dataset'

def convert_jsonl_to_bio(input_filename, output_filename):
    input_path = os.path.join(dataset_dir, input_filename)
    output_path = os.path.join(dataset_dir, output_filename)

    if not os.path.exists(input_path):
        print(f"File {input_filename} tidak ditemukan.")
        return

    bio_dataset = []

    with open(input_path, 'r', encoding='utf-8') as f:
        for line in f:
            data = json.loads(line)
```

```

tokens = data.get('tokens', [])
spans = data.get('spans', [])

# membuat daftar tag 'O' awal sepanjang jumlah token
tags = ['O'] * len(tokens)

# mencocokkan setiap posisi token dengan koordinat
karakter span
for span in spans:
    span_start = span.get('start')
    span_end = span.get('end')
    # bersihkan nama label (ambil aspek utamanya saja,
    misal PRICE, PRODUCT, PLACE, PROMOTION)
    raw_label = span.get('label', 'UNKNOWN').split('_')[0]

    is_first = True
    for idx, token in enumerate(tokens):
        t_start = token.get('start')
        t_end = token.get('end')

        # Cek apakah token berada di dalam jangkauan
        karakter span
        if t_start >= span_start and t_end <= span_end:
            if is_first:
                tags[idx] = f"B-{raw_label}"
                is_first = False
            else:
                # Jika token sebelumnya sudah B- atau I-,
                teruskan sebagai Inside (I-)
                tags[idx] = f"I-{raw_label}"

        # Simpan kata dan tag bionya ke dalam list
        words = [t.get('text') for t in tokens]
        bio_dataset.append({
            'sentence_tokens': words,
            'bio_tags': tags
        })

# Simpan hasil konversi ke file JSON baru untuk persiapan masuk ke
model deep learning
with open(output_path, 'w', encoding='utf-8') as out_f:
    json.dump(bio_dataset, out_f, ensure_ascii=False, indent=2)

    print(f"Berhasil mengonversi {input_filename} ->
    {output_filename}")
    return bio_dataset

print("Memulai proses pembuatan Tag BIO untuk data Train, Val, dan

```

```

Test...)
# menjalankan konversi untuk ketiga pecahan data secara konsisten
train_bio = convert_jsonl_to_bio('Kelp3_ner_train.jsonl',
    'Kelp3_bio_train.json')
val_bio = convert_jsonl_to_bio('Kelp3_ner_val.jsonl',
    'Kelp3_bio_val.json')
test_bio = convert_jsonl_to_bio('Kelp3_ner_test.jsonl',
    'Kelp3_bio_test.json')

# liat contoh hasil konversi baris pertama data Train
print("\nContoh Hasil Konversi Baris Pertama (Token & Tag BIO):")
if train_bio:
    example = train_bio[1] # liat baris kedua biar lebih kelihatan
    aspeknya
    for w, t in zip(example['sentence_tokens'][:15],
        example['bio_tags'][:15]):
        print(f"{w:<15} -> {t}")

```

Memulai proses pembuatan Tag BIO untuk data Train, Val, dan Test...
 Berhasil mengonversi Kelp3_ner_train.jsonl -> Kelp3_bio_train.json
 Berhasil mengonversi Kelp3_ner_val.jsonl -> Kelp3_bio_val.json
 Berhasil mengonversi Kelp3_ner_test.jsonl -> Kelp3_bio_test.json

Contoh Hasil Konversi Baris Pertama (Token & Tag BIO):

Pas	-> B-PLACE
buat	-> I-PLACE
emak	-> I-PLACE
emak	-> I-PLACE
yg	-> I-PLACE
suka	-> I-PLACE
shopping	-> I-PLACE
...	-> 0

```

import os
import json
import torch
import numpy as np
from torch.utils.data import Dataset
from transformers import AutoTokenizer,
    AutoModelForTokenClassification, TrainingArguments, Trainer,
    DataCollatorForTokenClassification
from evaluate import load

```

1. KONFIGURASI PATH & SKEMA LABEL MINIMAL

```

dataset_dir = '../dataset'
model_checkpoint = "indobenchmark/indobert-base-p1"

```

Skema tag NER minimal

```

label_list = ['O', 'B-PRODUCT', 'I-PRODUCT', 'B-PRICE', 'I-PRICE', 'B-PLACE', 'I-PLACE', 'B-PROMOTION', 'I-PROMOTION']

```

```

label2id = {l: i for i, l in enumerate(label_list)}
id2label = {i: l for i, l in enumerate(label_list)}

print("Loading Tokenizer IndoBERT...")
tokenizer = AutoTokenizer.from_pretrained(model_checkpoint,
do_lower_case=True)

# 2. FUNGSI LOAD DATA BIO JSON
def load_bio_json(filename):
    with open(os.path.join(dataset_dir, filename), 'r', encoding='utf-8') as f:
        return json.load(f)

train_data = load_bio_json('Kelp3_bio_train.json')
val_data = load_bio_json('Kelp3_bio_val.json')

# 3. PROSES ALIGNMENT TOKEN-LABEL (SUBWORD HANDLING)
def align_and_tokenize(data_list):
    tokenized_inputs = []

    for item in data_list:
        words = item['sentence_tokens']
        labels = item['bio_tags']

        # Tokenisasi menggunakan tokenizer bawaan IndoBERT
        inputs = tokenizer(words, is_split_into_words=True,
truncation=True, max_length=128)

        # Mapping token hasil potongan subwords kembali ke ID label
asli
        word_ids = inputs.word_ids()
        label_ids = []

        for word_idx in word_ids:
            if word_idx is None:
                # tag -100 untuk token spesial ([CLS], [SEP]) agar
diabaikan oleh fungsi loss PyTorch
                label_ids.append(-100)
            else:
                label_name = labels[word_idx]
                # Jika tag tidak terdaftar di skema minimal, amankan
sebagai '0'
                label_ids.append(label2id.get(label_name, 0))

        inputs["labels"] = label_ids
        tokenized_inputs.append(inputs)

    return tokenized_inputs

print("Menyelaraskan token subword IndoBERT dengan label tag BIO...")

```

```

train_tokenized = align_and_tokenize(train_data)
val_tokenized = align_and_tokenize(val_data)

# 4. MEMBUAT PYTORCH DATASET CLASS
class NERDataset(Dataset):
    def __init__(self, tokenized_data):
        self.data = tokenized_data
    def __len__(self):
        return len(self.data)
    def __getitem__(self, idx):
        return {k: torch.tensor(v) for k, v in self.data[idx].items()}

train_dataset = NERDataset(train_tokenized)
val_dataset = NERDataset(val_tokenized)

# 5. SETUP METRIK EVALUASI (SEQUEVAL)
metric = load("sequeval")
def compute_metrics(p):
    predictions, labels = p
    predictions = np.argmax(predictions, axis=2)

    true_predictions = [
        [label_list[p] for (p, l) in zip(prediction, label) if l != -
100]
        for prediction, label in zip(predictions, labels)
    ]
    true_labels = [
        [label_list[l] for (p, l) in zip(prediction, label) if l != -
100]
        for prediction, label in zip(predictions, labels)
    ]
    results = metric.compute(predictions=true_predictions,
references=true_labels)
    return {
        "precision": results["overall_precision"],
        "recall": results["overall_recall"],
        "f1": results["overall_f1"],
        "accuracy": results["overall_accuracy"],
    }

# 6. KONFIGURASI ARSITEKTUR INDOBERT TOKEN CLASSIFICATION
print("Mengunduh & Mengonfigurasi Arsitektur IndoBERT di GPU...")
model = AutoModelForTokenClassification.from_pretrained(
    model_checkpoint,
    num_labels=len(label_list),
    id2label=id2label,
    label2id=label2id
)

# 7. CONFIG TRAINING ARGUMENTS (DENGAN EARLY STOPPING/TUNING)

```

```

training_args = TrainingArguments(
    output_dir="./results",
    eval_strategy="epoch",
    save_strategy="epoch",
    learning_rate=3e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=5,           # Latih sebanyak 5 iterasi penuh
    weight_decay=0.01,
    logging_dir="./logs",
    logging_steps=10,
    load_best_model_at_end=True,  # Otomatis mengunci model terbaik
    # di akhir epoch (Tuning)
    metric_for_best_model="f1",
    report_to="none"
)

```

```

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    processing_class=tokenizer,
    data_collator=DataCollatorForTokenClassification(tokenizer),
    compute_metrics=compute_metrics
)

```

```

print("\nMemulai Proses Pelatihan Model NER IndoBERT...")
trainer.train()
print("Pelatihan Selesai! Model terbaik telah disimpan otomatis.")

```

Loading Tokenizer IndoBERT...

Menyelaraskan token subword IndoBERT dengan label tag BIO...

Mengunduh & Mengonfigurasi Arsitektur IndoBERT di GPU...

[transformers] You passed `num\_labels=9` which is incompatible to the `id2label` map of length `5`.

```

{"model_id": "972c68a772044bfbafa248a94c546d26", "version_major": 2, "version_minor": 0}

```

[transformers] BertForTokenClassification LOAD REPORT from: indobenchmark/indobert-base-pl

Key	Status
pooler.dense.weight	UNEXPECTED
pooler.dense.bias	UNEXPECTED
classifier.weight	MISSING
classifier.bias	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
 - MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
- [transformers] `logging\_dir` is deprecated and will be removed in v5.2. Please set `TensorBoard.LoggingDir` instead.

Memulai Proses Pelatihan Model NER IndoBERT...

<IPython.core.display.HTML object>

```
{"model_id": "ae4755d87b3a4a15acc233f3651f8dbd", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "5be04f3c4c16451a906ecfe29f202190", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "8f599acb770742b1bae2ba0f432a1067", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "8ff8c52237e945138b2aa1037065d6bf", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "0dc6d0e956684634854c662e3f973011", "version_major": 2, "version_minor": 0}
```

Pelatihan Selesai! Model terbaik telah disimpan otomatis.

```
from sequeval.metrics import classification_report as sequeval_report
```

```
print("Memproses dan menyelaraskan data Test Set...")
```

```
# 1. Load dan tokenisasi data test
```

```
test_data = load_bio_json('Kelp3_bio_test.json')
```

```
test_tokenized = align_and_tokenize(test_data)
```

```
test_dataset = NERDataset(test_tokenized)
```

```
print("Melakukan prediksi model terbaik pada Test Set...")
```

```
# 2. Ambil hasil prediksi dari trainer
```

```
predictions, labels, _ = trainer.predict(test_dataset)
```

```
predictions = np.argmax(predictions, axis=2)
```

```
# 3. Konversi ID angka kembali menjadi teks Tag BIO (abaikan tag -100)
```

```
true_predictions = [  
    [label_list[p] for (p, l) in zip(prediction, label) if l != -100]  
    for prediction, label in zip(predictions, labels)  
]
```

```
true_labels = [  
    [label_list[l] for (p, l) in zip(prediction, label) if l != -100]  
    for prediction, label in zip(predictions, labels)  
]
```

```

print("\n=====")
print("CLASSIFICATION REPORT PER ENTITAS (PROYEK B - NER)")
print("=====")
# Menggunakan sequeval_report untuk mengevaluasi performa kelompok
frasa entitas secara utuh
print(sequeval_report(true_labels, true_predictions, zero_division=0))

# 4. KODE ANALISIS KESALAHAN (ERROR ANALYSIS NER)
print("\nMEMULAI ANALISIS KESALAHAN MODEL NER (CONTOH KASUS SALAH
TAG):")
print("=====
=====")

error_count = 0
for idx in range(len(test_data)):
    if error_count >= 2: # tampilkan 2 contoh kasus saja agar ringkas
        break

    t_label = true_labels[idx]
    p_label = true_predictions[idx]

    # Jika ada perbedaan tebakan tag di dalam kalimat
    if t_label != p_label:
        words = test_data[idx]['sentence_tokens']
        print(f"\n□ Contoh Salah {error_count+1}:")
        print(f"{'Kata':<12} | {'Label Asli':<12} | {'AI Menebak':<12}
| Status")
        print("-" * 55)

        # Menggabungkan token kata, label asli, dan hasil prediksi
secara aman
        for w, tl, pl in list(zip(words, t_label, p_label))[:12]:
            status = "□" if tl == pl else "□"
            print(f"{'w':<12} | {'tl':<12} | {'pl':<12} | {'status'}")

        print("-" * 55)
        error_count += 1

```

Memproses dan menyelaraskan data Test Set...

Melakukan prediksi model terbaik pada Test Set...

<IPython.core.display.HTML object>

```

=====
CLASSIFICATION REPORT PER ENTITAS (PROYEK B - NER)
=====
precision    recall  f1-score   support

```

PLACE	0.33	0.39	0.36	332
PRICE	0.52	0.54	0.53	122
PRODUCT	0.47	0.53	0.50	1101
PROMOTION	0.00	0.00	0.00	42
micro avg	0.43	0.49	0.46	1597
macro avg	0.33	0.36	0.34	1597
weighted avg	0.43	0.49	0.46	1597

MEMULAI ANALISIS KESALAHAN MODEL NER (CONTOH KASUS SALAH TAG):

□ Contoh Salah 1:

Kata	Label Asli	AI Menebak	Status
Sangat	B-PRODUCT	B-PLACE	❌
bagus	I-PRODUCT	I-PLACE	❌
dan	0	0	❌
Rama	B-PLACE	B-PLACE	❌
...	0	0	❌

□ Contoh Salah 2:

Kata	Label Asli	AI Menebak	Status
Belajar	B-PRODUCT	0	❌
bikin	I-PRODUCT	I-PRODUCT	❌
gerabah	I-PRODUCT	I-PRODUCT	❌
bisa	I-PRODUCT	I-PRODUCT	❌
di	I-PRODUCT	0	❌
sini	I-PRODUCT	0	❌

```
import os
```

```
ner_model_dir = '../models/Kelp3_best_indobert_ner'
os.makedirs(ner_model_dir, exist_ok=True)
```

```
print("Menyimpan Model IndoBERT NER dan Tokenizer ke folder  
models...")
```

```
# Simpan weights model dan konfigurasi tokenizer secara permanen
```

```
trainer.save_model(ner_model_dir)
```

```
tokenizer.save_pretrained(ner_model_dir)
```

```
print("Model dan Tokenizer berhasil disimpan di folder:",  
ner_model_dir)
```

Menyimpan Model IndoBERT NER dan Tokenizer ke folder models...

```
{"model_id": "ebad4c82be2042a28a9aaa7c14947ee5", "version_major": 2, "version_minor": 0}
```

Model dan Tokenizer berhasil disimpan di folder:
../models/Kelp3_best_indobert_ner